Module 4 -- Arduino Microcontroller

# Lesson 32: Piezo Buzzer and Tones -- Give Your Arduino a Musical Voice

★ Star-3 Intermediate ⏰ 40--45 minutes 👤 Age 6--12 years ⚡ 250 XP

## Your Mission Today

Circuit Explorer, your Arduino can blink, fade, read buttons, and scan LEDs. But it has been totally silent! Today that changes. You will connect a **piezo buzzer** and use the `tone()` function to play real musical notes. By the end of this lesson, you will have programmed your own melody — maybe even "Twinkle Twinkle Little Star" or "Happy Birthday." Time to make some noise!

## Learning Objectives

By the end of this lesson, you will be able to:

- Connect a piezo buzzer to an Arduino digital pin
- Use the `tone()` and `noTone()` functions to play sounds
- Understand that musical notes are specific frequencies (measured in Hz)
- Program a simple melody using note frequencies and durations
- Combine a button with the buzzer to play notes on demand

## What You Need

| Item | Qty |
|---|---|
| Arduino Uno + USB cable | 1 |
| Breadboard | 1 |
| Piezo buzzer (passive type) | 1 |
| Push button | 1 |
| Jumper wires | 4 |
| Multimeter (Magic Measurement Wand!) | 1 |

## How to Teach This Lesson

### Step 1: Hook — How Does Sound Work? (3 min)

💬

"Put your hand on your throat and hum. Feel that vibration? That vibration makes the air shake, and your ears pick up the shaking air as sound."

💬

"A buzzer works the same way! A tiny disc inside it vibrates really fast, pushing the air back and forth. The FASTER it vibrates, the HIGHER the sound. The SLOWER it vibrates, the LOWER the sound."

**Key Vocabulary:**

- **Frequency** — how many times per second something vibrates. Measured in **Hertz (Hz)**
- **440 Hz** means 440 vibrations per second. This is the musical note **A** (above middle C)

**Award:** **+10 XP** **for learning how sound works!**

## Step 2: Wire the Buzzer (5 min)

**Important — Use a PASSIVE buzzer for tone(), not an active buzzer.**

> "A passive buzzer needs a signal to make sound (the Arduino provides it with tone()). An active buzzer has its own built-in oscillator and just needs power. We want passive so WE control the pitch."

**Wiring:**

```
Arduino Uno                    Breadboard
+---------+
| Pin 8   |----[wire]----+---- Buzzer (+) leg (longer)
|         |
| GND     |----[wire]----+---- Buzzer (-) leg (shorter)
+---------+

Buzzer Detail:
+--------+
|  (+)   |  <-- longer leg or marked with +
|   o    |
|  (-)   |  <-- shorter leg
+--------+
```

**Steps:**

1. Place the buzzer on the breadboard
2. Connect the **positive leg** (+, often longer or marked) to **Arduino pin 8**
3. Connect the **negative leg** (-) to **Arduino GND**

**Award:** **+10 XP** **for wiring the buzzer!**

## Step 3: Play Your First Tone (5 min)

```
tone(pin, frequency, duration);
```

| Parameter | What It Means | Example |
|---|---|---|
| `pin` | Which pin the buzzer is on | 8 |
| `frequency` | How many Hz (pitch) | 440 (note A) |
| `duration` | How long in milliseconds | 500 (half second) |

**First Tone Code:**

```
int buzzerPin = 8;

void setup() {
  tone(buzzerPin, 440, 1000);  // Play A4 for 1 second
}

void loop() {
  // Nothing here
}
```

"Upload this and you should hear a single beeeep! That is the note A at 440 Hz — the same note orchestras use to tune their instruments."

**Award:** **+10 XP** **for playing your first tone!**

## Step 4: The Musical Note Chart (5 min)

Each musical note has a specific frequency:

```
Note Frequencies (4th Octave):

  C4  = 262 Hz     C#4 = 277 Hz
  D4  = 294 Hz     D#4 = 311 Hz
  E4  = 330 Hz
  F4  = 349 Hz     F#4 = 370 Hz
  G4  = 392 Hz     G#4 = 415 Hz
  A4  = 440 Hz     A#4 = 466 Hz
  B4  = 494 Hz
  C5  = 523 Hz     (one octave higher)
  D5  = 587 Hz
  E5  = 659 Hz
```

💬

"Higher numbers = higher pitch. C4 (262) sounds lower than C5 (523). C5 is exactly double C4 — that is what 'one octave' means!"

**Award:** **+10 XP** **for learning note frequencies!**

## Step 5: Program a Melody (10 min)

**Twinkle Twinkle Little Star:**

```
int buzzerPin = 8;

// Note frequencies
int C4 = 262;
int D4 = 294;
int E4 = 330;
int F4 = 349;
int G4 = 392;
int A4 = 440;

// The melody: Twinkle Twinkle Little Star
int melody[] = {C4, C4, G4, G4, A4, A4, G4,
                F4, F4, E4, E4, D4, D4, C4};

int noteDurations[] = {400, 400, 400, 400, 400, 400, 800,
                       400, 400, 400, 400, 400, 400, 800};

int numNotes = 14;

void setup() {
  Serial.begin(9600);
  Serial.println("Playing Twinkle Twinkle Little Star!");

  for (int i = 0; i < numNotes; i++) {
    tone(buzzerPin, melody[i], noteDurations[i]);
    delay(noteDurations[i] + 50);  // Small gap between notes
    noTone(buzzerPin);
  }

  Serial.println("Song complete!");
}

void loop() {
  // Nothing -- plays once in setup
}
```

"You just used arrays AND for loops together to play a real song! The melody array holds the notes, the noteDurations array holds how long each note plays, and the for loop steps through them one by one."

**Award:** **+30 XP** **for programming a melody!**

Lesson 32: Piezo Buzzer and Tones -- Give Your Arduino a Musical Voice

Module 4: Arduino Microcontroller

## Step 6: Button-Triggered Sound Effects (5 min)

**Add a button to play sounds on demand:**

```
int buzzerPin = 8;
int buttonPin = 2;

void setup() {
  pinMode(buttonPin, INPUT_PULLUP);
}

void loop() {
  if (digitalRead(buttonPin) == LOW) {
    // Play a fun sound effect when button is pressed
    tone(buzzerPin, 262, 100); delay(150);
    tone(buzzerPin, 330, 100); delay(150);
    tone(buzzerPin, 392, 100); delay(150);
    tone(buzzerPin, 523, 300); delay(350);
    noTone(buzzerPin);
  }
}
```

> "Press the button and you hear a little victory jingle! This is exactly how video game sound effects work."

**Award:** **+20 XP** **for button-triggered sounds!**

## Step 7: Wand Check — What Does a Tone Look Like Electrically? (5 min)

> "The tone() function makes pin 8 switch between 5V and 0V really fast — at the exact frequency of the note. Let us see what the Wand shows!"

1. Set the Wand to **DC Voltage**
2. Black probe on **GND**, red probe on **pin 8**
3. Play a continuous tone: `tone(8, 440);` in setup with nothing in loop

> 💬 "The Wand shows an average voltage around 2.5V. Just like PWM, the pin is rapidly switching between 5V and 0V. The buzzer vibrates with each switch, making sound!"

**Fun Experiment:**

- Play a low note (262 Hz): Wand shows ~2.5V
- Play a high note (1000 Hz): Wand still shows ~2.5V
- The average voltage is the same, but the SPEED of switching is different!

**Award:** **+20 XP** **for measuring tone voltage!**

---

## Quick Quiz — Earn Bonus XP!

**Question 1:** What does the `tone()` function do?

- A) Measures sound volume
- B) Makes a buzzer vibrate at a specific frequency
- C) Records your voice

**(Correct: B —** **+20 XP** **!)**

**Question 2:** The note A4 has a frequency of 440 Hz. What does Hz mean?

- A) Vibrations per minute
- B) Vibrations per second
- C) Vibrations per hour

**(Correct: B —** **+20 XP** **!)**

**Question 3:** To play a higher-pitched note, do you use a higher or lower frequency number?

- A) Higher number = higher pitch
- B) Lower number = higher pitch
- C) The number does not matter

## Lesson 32 Complete!

```
==============================================

   MELODY MAKER BADGE UNLOCKED!

   Skills unlocked:
   [check] Wire a piezo buzzer
   [check] Use tone() and noTone()
   [check] Know note frequencies
   [check] Program a melody with arrays
   [check] Trigger sounds with a button

==============================================
```

**XP Breakdown:**

| Activity | XP |
|---|---|
| Hook — sound concepts | 10 |
| Buzzer wiring | 10 |
| First tone | 10 |
| Note frequency chart | 10 |
| Twinkle Twinkle melody | 30 |
| Button sound effects | 20 |
| Wand Check | 20 |
| Quiz (3 questions) | 60 |
| **TOTAL POSSIBLE** | **250** |

## Coming Up Next...

In **Lesson 33**, you will add a **screen** to your Arduino! The **16x2 LCD display** lets your Arduino show text, numbers, and messages. No more relying on the Serial Monitor — your gadget will have its own display. Get ready to see "Hello World!" on a real screen!

## Troubleshooting

| Problem | Fix |
|---|---|
| No sound at all | Make sure you have a PASSIVE buzzer (not active); check wiring polarity |
| Sound is very quiet | Some buzzers are quiet — try holding it closer to your ear, or try a different buzzer |
| Wrong notes playing | Double-check the frequency values in your melody array |
| tone() gives a compile error | Make sure you are passing 2 or 3 arguments: pin, frequency, and optionally duration |
| Buzzer keeps buzzing after sketch ends | Add `noTone(buzzerPin);` at the end of your melody to stop it |